Final Report: Cloud-Deployed Travel Assistant Chatbot

Student ID: 25402005

Student name: Huang Qiaolin

ID of my chatbot: @linsuijian_bot

1. Project Overview

I built a Telegram chatbot that works as a personal travel assistant for Hong Kong. Users can ask for attraction recommendations or check real-time weather. The bot understands natural language and replies with useful, detailed information.

The backend uses the HKBU GenAI API (model gpt-5-mini) to generate answers. For weather queries, it calls the Hong Kong Observatory open data API to get accurate forecasts. The whole system is packaged in Docker, runs on an AWS EC2 instance, and uses GitHub Actions for automatic deployment.

2. Key Features

Attraction recommendation

When a user asks for places to visit, the bot replies with categories such as landmarks, food, nature, and shopping. It also gives transport tips and practical advice. The answer is generated entirely by the LLM, so no external database of attractions is needed.

Real-time weather query

If the user asks about weather (e.g., “How is the temperature in Mong Kok East?”), the bot detects weather keywords, fetches the 9-day forecast from the Hong Kong Observatory, and injects the real data into the LLM prompt. The model then answers based on facts, not its own knowledge. This prevents hallucinations and gives users reliable numbers like “23–29°C”.

Personalised memory (innovation)

I added a small but useful feature: the bot remembers what the user likes. If someone says “I like hiking”, the bot saves this preference into a local SQLite database. Next time the same user asks for attraction recommendations, the bot

prioritises hiking-related places (e.g., Dragon's Back, Lion Rock). All conversations are also logged in the same database, which satisfies the project's requirement of using a database system for logging.

3. Cloud and DevOps Implementation

Docker

I wrote a Dockerfile based on python:3.12-slim. It copies the code, installs dependencies, and exposes port 8080 for health checks. The image is built and pushed to Docker Hub as linsuijian/travel-bot:latest.

CI/CD with GitHub Actions

Every time I push code to the main branch, a GitHub Actions workflow runs. It SSHs into my EC2 instance, pulls the latest Docker image, stops the old container, and starts a new one with the same name. This means I never have to manually log into EC2 to update the bot.

Observability

Health check: A simple HTTP server runs on port 8080 inside the container. Visiting http://<EC2_IP>:8080/health returns OK. This tells me the bot is alive.

Structured logs: For each user message, the bot prints a JSON log entry containing user_id, user_message, response_time_ms, and whether it was a weather query. I can view these logs with docker logs travel-bot.

Cost awareness

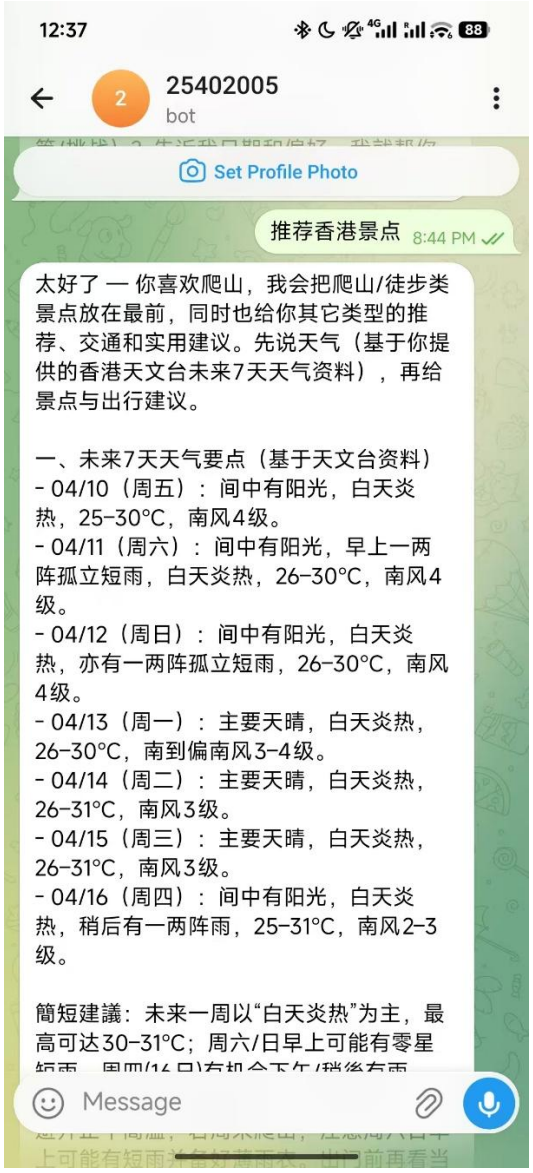
My EC2 instance is a t3.micro, which is within the AWS Free Tier. When I am not developing or demonstrating, I keep the instance stopped. The only cost is a small amount for EBS storage (a few cents per month). In the future, I could set up an automatic start/stop schedule to save even more.

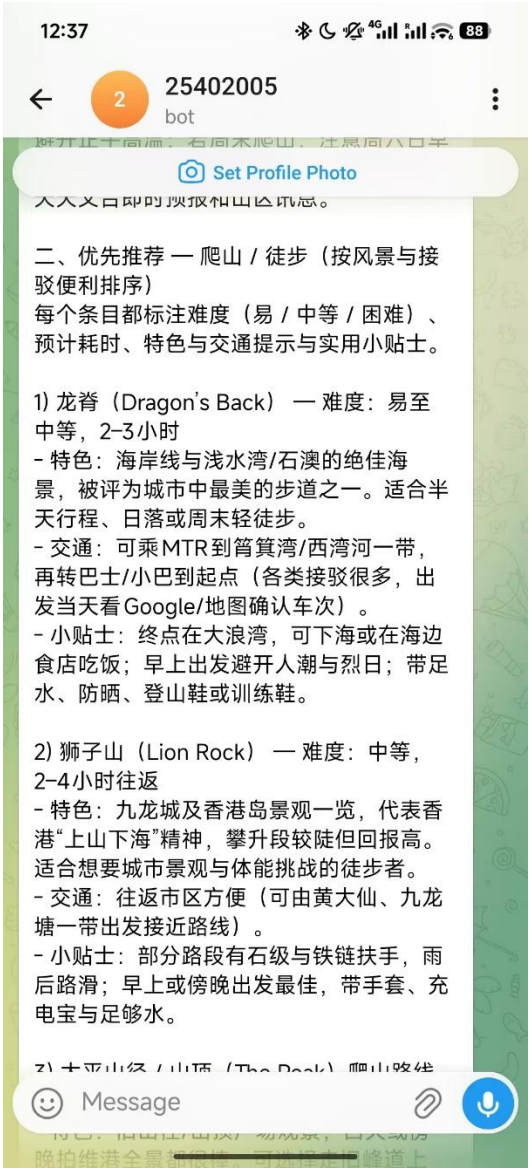
4. Demonstration Evidence

I have taken screenshots of the following to show during the presentation:

4.1 Telegram conversation showing attraction recommendation (detailed categories)

4.2 Telegram conversation showing weather query with real temperature data & Telegram conversation showing interest memory (“I like hiking” → later recommendation prioritises hiking)



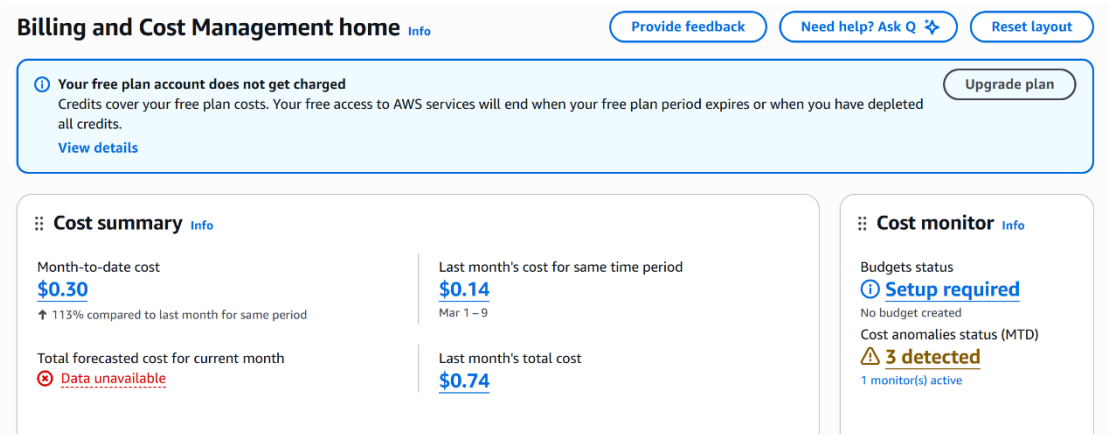


4.3 curl http://localhost:8080/health returning OK

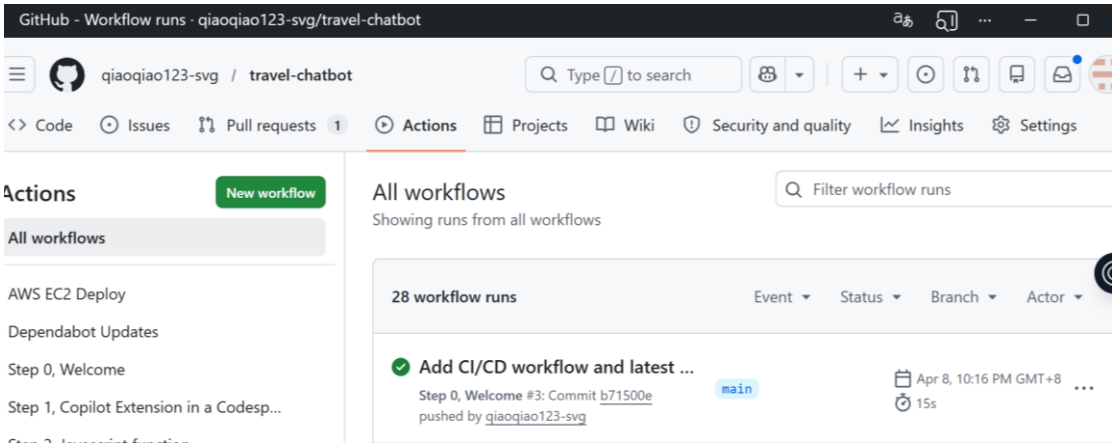
```
[ec2-user@ip-172-31-23-54 ~]$ curl http://localhost:8080/health
OK[ec2-user@ip-172-31-23-54 ~]$
```

4.4 EC2 instance state = “stopped” & billing of it (cost control)

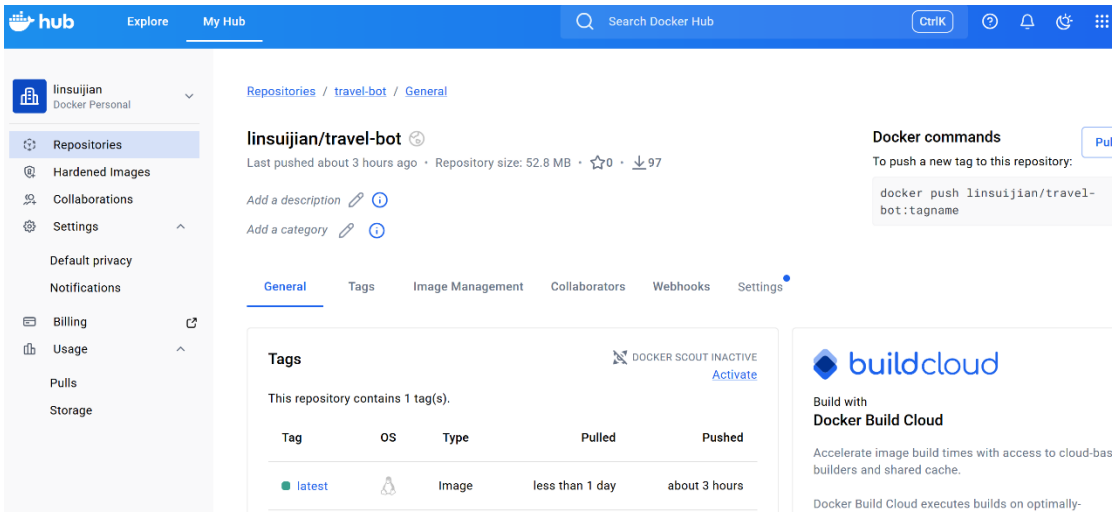
实例 ID	实例状态	实例类型
i-0b26bf62f8ab1c56d	已停止	t3.micro



4.5 GitHub Actions workflow with a green check mark



4.6 Docker Hub repository page showing linsuijian/travel-bot:latest



5. Links

GitHub repository: https://github.com/qiaoqiao123-svg/travel-chatbot

Docker Hub image: linsuijian/travel-bot:latest